

1 MID-YEAR REPORT

1.1 Research Plan

This research investigates the design and implementation of a reconfigurable neural processing unit (NPU) for accelerating convolutional neural networks (CNN) and energy efficiency on an FPGA. The increasing computational demand in machine learning in this day and age, particularly deep CNNs, present a challenge when deployed at the edge, where hardware resources are limited. Conventional hardware accelerators lack flexibility to adapt to diverse model structures; ultimately motivating the exploration of reconfigurable architectures that can dynamically adjust to meet the efficiency requirements.

The foundation of this research is the development of a scalable systolic array architecture implemented in VHDL, designed to perform matrix multiplication at the heart of CNN computations, eventually becoming useful for applications like object recognition and tracking. Systolic arrays provide a basic structure and high throughput, making them suitable for FPGA fabrics and a good choice for optimising multiply accumulate (MAC) operations in an NPU. However, conventional systolic arrays pose an issue. Since most of their implementations assume fixed dimensions and dense matrices, they limit their efficiency when it comes to applying to CNNs with sparsity in weights and activations. To address this, we aim to extend the conventional systolic array design with a hardware control unit capable of dynamically configuring the active processing elements, and with mechanisms to properly stagger and align inputs to the array for computation.

A key part of the research is to understand how sparsity impacts the performance and efficiency of systolic arrays, due to techniques introduced into CNNs such as the ReLU activation and weight pruning. While sparsity has the potential to reduce computation and energy consumption by skipping the zero-value elements, basic systolic array architectures fail to exploit this because all processing elements remain active regardless of the data value. This motivates the integration of sparsity-aware mechanisms.

The current design approach follows a bottom-up methodology, starting with a functioning systolic array and hardware control unit capable of supporting arbitrary square array sizes up to a maximum limit, with inputs and other configurations driven by a softcore processor, NIOS II. The inclusion of this processor enables a higher level of flexibility, allowing the system to adapt to different models whilst maintaining a high throughput, ultimately balancing the need for efficiency with the ability to handle variability in CNN architectures.

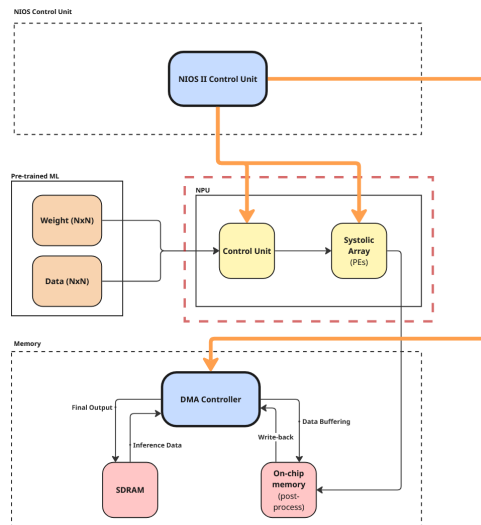


Figure 1: System Overview

1.2 Current Implementation

At the current stage of the project, the focus has been on the MAC engine implemented as a systolic array. This implementation serves as the computational core for convolutional neural network (CNN) inference, where the operations are multiply-accumulate over matrices of activations and weights. The current system is deliberately modular to support scalability and facilitate future extensions toward sparsity-aware and dynamically reconfigurable architecture.

1.2.1 Systolic Array and Processing Element

The systolic array has been implemented in hardware using VHDL as a two-dimensional network of processing elements (PEs). Each PE is responsible for computing and accumulating its own local partial sum of the output matrix. Data and weights are propagated through the array in a pipeline where data moves horizontally and weights move vertically, while each PE multiplies the local data and weight inputs and accumulates the results. This "local" accumulation is internal to each PE, avoiding incorrect accumulation to propagate across the array. The array in the current implementation supports square matrices up to 8x8, with the dimension parametrised for flexibility.

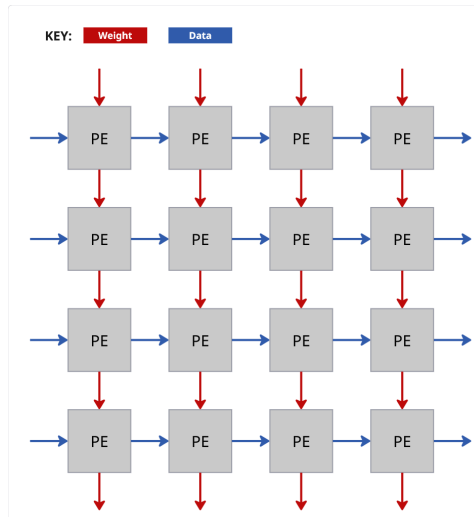


Figure 2: 4x4 Systolic array example

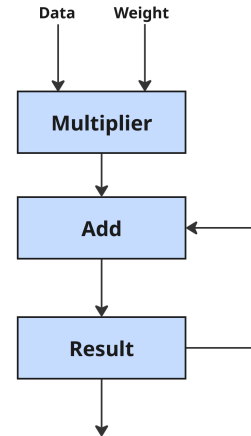


Figure 3: Multiply-Accumulate (MAC) Operation Hardware resources

1.2.2 Control Unit

A dedicated hardware control has been established for feeding the array with the appropriate data and weight streams and enabling only the relevant PEs for a given matrix size. The control unit generates the staggered input patterns required by the systolic array's computation, to produce and shift streams of rows of the input activation matrix and columns of the weight matrix, such that they arrive at the correct PEs at the correct clock cycles. This staggered feeding is implemented using shift-register logic with two shift-register-based signals: data shift and weight shift, inside the control unit. So far, this is the best scalable method to align the streams within the flow of the array. The control unit can also generate a PE enable mask that dynamically enables or disables the PEs depending on the configured array size, which is essential for supporting smaller matrices on the same hardware fabric without wasting power on unused elements. The mask itself is generated per cycle using a loop that enables only those PEs within the active region of the array.

1.2.3 Top-Level Module

The current design is integrated with a top-level VHDL module that connects the control unit and the systolic array, in order to create a simple interface of input matrices, start/reset signals, and output matrix results. The top level wires the shift register outputs and PE enable mask from the control unit directly into the systolic array. Additionally, the current top-level entity enables a fast scalable interface for the upcoming additions for memory-mapped integration and the external softcore controller run by NIOS II.

1.2.4 Testing Performance

A testbench has been written to verify functionality across matrix sizes from 1x1 up to 8x8, showing correct operation and correct accumulation of partial sums in the expected number of clock cycles. The same testbench and code was utilised to test different sizes using the same hardware configuration, essentially relying on the control unit to configure the PE enable mask, effectively demonstrating reconfigurability. As shown in figure 4 and figure 5, the testbench output displays a 4x4 resultant matrix that is verified by the MATLAB script output.

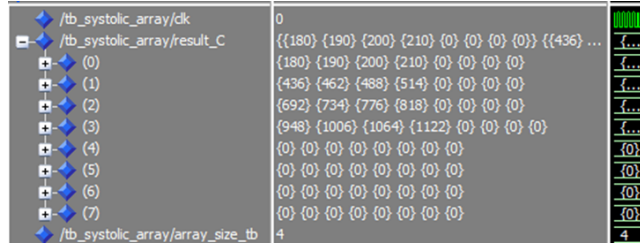


Figure 4: 4x4 Modelsim simulation outcome

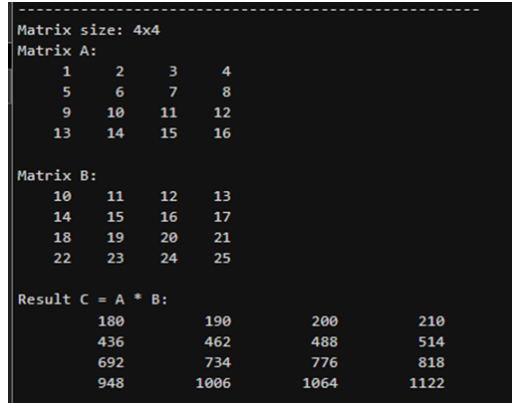


Figure 5: 4x4 Matrix Multiplication output from MATLAB script

In figure 6, quartus has assumed that the system will be running at the highest speed, ultimately failing the timing requirements and causing compilation errors. To mitigate this, a constraints.sdc file was produced to set up a 50Mhz clock, allowing a compiled system and working timing analyser, as shown in 7. According to 7, 118.64 Mhz is the safe operating clock frequency without violating any constraints (worse case). To conclude, this frequency is more than sufficient for the current systolic array implementation, allowing headroom for more reconfiguration logic and softcore processing.

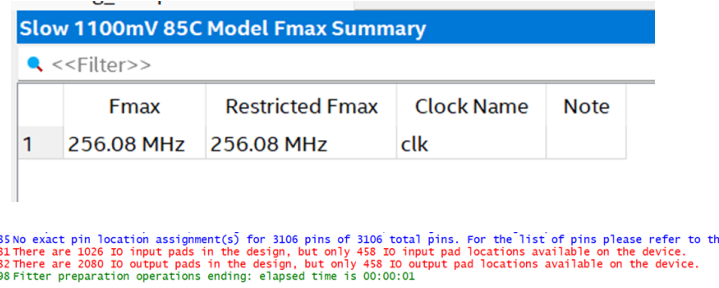


Figure 6: Compilation error, producing Fmax of 256Mhz

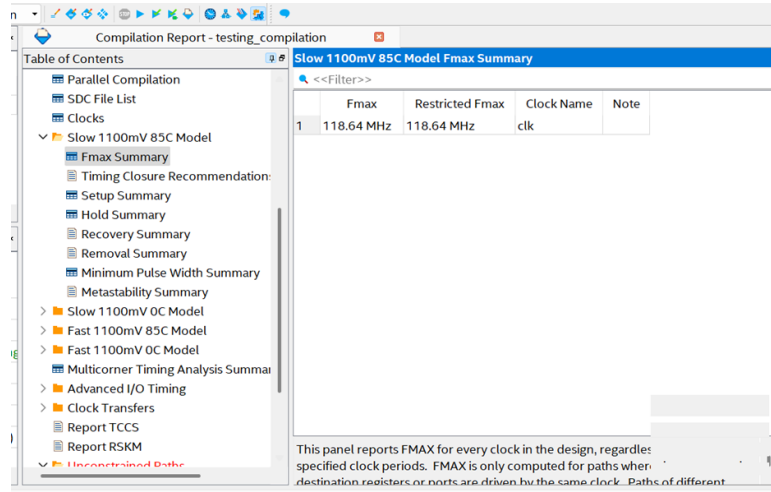


Figure 7: System Overview

1.3 Future Implementation

1.3.1 Softcore Processor Integration

The next major milestone involves integrating a softcore processor such as NIOS II into the design. This processor will act as the software-controlled component of the system, coordinating input/output movement, handling reconfigurability commands and manage the sequence and scheduling during runtime.

The processor will interface with the hardware-accelerated systolic array through a memory-mapped interface, allowing high-level software programs to be written, potentially in C using hardware abstraction layer APIs. This can be used to:

- Load Matrices into shared memory or input buffers. This could be for example, loading weights from any external memory to on-chip memory. BRAM would preload the weights and feed them column by column into the weight shift registers. NIOS can program the Direct Memory Access (DMA) controller without the need for CPU intervention and the currently implemented shift registers in the datapath pipeline will take care of the rest.
- Trigger systolic array computations
- Monitor results and status flags
- Dynamically change array configuration parameters.

1.3.2 Reconfigurability and Sparsity Handling

To meet the aim of a reconfigurable NPU, the hardware must support dynamic array scaling where it can allow matrix sizes smaller than 8x8 via a PE enable mask, and it must also have sparsity awareness, which is by introducing a skipping logic into PEs that disable MAC operations when input is zero, reducing resource consumption.

In addition to the run-time skip mechanism, the system will be designed to accommodate more sparsity exploitation methods. During the pre-processing stage, pruning can be applied to eliminate the insignificant weights and activations, reducing the number of actual computations required. Compressed formatting be utilised to represent nonzero elements. This is where the main controller can schedule only the valid computation pairs into the systolic array, avoiding the zero values. This will allow the system to scale efficiently with sparsity as the main issue, reducing both computation and staying within memory limitations.

1.3.3 Direct Memory Access Controller and On-chip Communication

To reduce CPU overhead, the soft-core processor will be assisted by a DMA controller that transfers matrix data between memory and the accelerator. This will allow overlapping of data movement and computation.

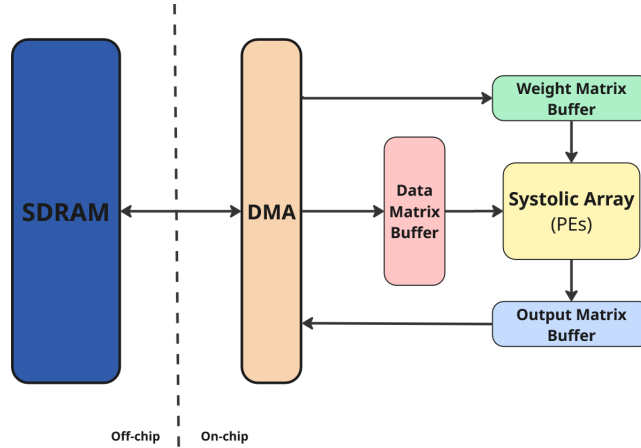


Figure 8: Memory Architecture Overview